

MODULE II

PROMPT ENGINEERING AND CORE ARCHITECTURES

WHAT IS A PROMPT?

- A prompt is the **input** provided to a generative AI model to guide its response.
- A prompt may contain:
 - Instructions
 - Questions
 - Context
 - Examples
 - Constraints
 - Input data
 - Desired output format
- **Example**
"Explain recursion to a first-year CSE student using a factorial example. Include Python code and explain the time complexity."
- This isn't simply a question. It contains several instructions about the desired response.

WHAT IS PROMPT ENGINEERING?

- Prompt engineering is the **systematic process** of **designing, testing, evaluating, and refining** prompts to obtain **useful and reliable** outputs from generative AI systems.
- Think of it as:

REQUIREMENT → PROMPT → AI OUTPUT → EVALUATION → PROMPT REFINEMENT → IMPROVED OUTPUT

- Prompt engineering is not: "Finding a magic sentence that always gives the perfect answer."
- It is an **iterative engineering activity**.

WHY DOES PROMPT ENGINEERING MATTER?

- Consider this prompt: **"Explain databases."**
- It is ambiguous.
- Now consider:

"Explain database normalization to first-year CSE students. Cover 1NF, 2NF and 3NF using a student-course database example. Include a comparison table and three common mistakes."

- The second prompt specifies:
 - Topic
 - Audience
 - Scope
 - Example
 - Output format
 - Additional requirements
- The clearer the task specification, the easier it is for the model to produce an appropriately targeted response.

ANATOMY OF A GOOD PROMPT/IMPORTANT ELEMENTS OF A GOOD PROMPT

- A practical prompt can contain:

ROLE + TASK + CONTEXT + CONSTRAINTS + OUTPUT FORMAT + EXAMPLES

Not every prompt needs all six.

- Example

Role: You are a CSE instructor.

Task: Explain binary search.

Context: Students understand arrays but not algorithm analysis.

Constraints: Use simple language and one example.

Output: Explanation → algorithm → code → complexity.

Example: Trace the search for the value 23

CLEAR AND SPECIFIC INSTRUCTIONS

- Weak prompt: **"Write something about operating systems."**
- Improved: **"Explain process scheduling in operating systems."**
- Strong prompt: **"Explain CPU scheduling algorithms to third-year CSE students. Cover FCFS, SJF, Priority Scheduling and Round Robin. For each algorithm, provide its working principle, one example, advantages, limitations, and time-related considerations."**
- Avoid vague verbs such as: Discuss, Tell me about, Write something, Explain everything unless the scope is clear.

CONTEXT

- Context tells the model what it needs to know before performing the task.
- Example: **"Explain recursion."**

OR

- **"Explain recursion to students who already understand loops and functions but struggle with recursive thinking."**
- The second prompt provides information about Audience, Existing knowledge, Learning difficulty
- Other useful context include Purpose, Domain, Input data, Previous conversation, User constraints

AUDIENCE SPECIFICATION

- The same concept can require different explanations depending on the audience
- Prompt A: **"Explain transformers to a school student."**
- Prompt B: **"Explain transformers to a third-year CSE student who knows neural networks, matrices, and probability."**
- Prompt C: **"Explain transformers to an NLP researcher."**
- The underlying concept is the same.
- The depth, terminology, assumptions, and examples should differ.

CONSTRAINTS

- Constraints specify limitations or requirements.
- Common constraints include Word count, Number of examples, Difficulty, Programming language, Topics to include, Topics to exclude, Tone, Output structure
- Example

"Explain attention mechanisms in approximately 500 words. Assume students know matrix multiplication but not attention. Include one numerical example and one diagram description."

- Constraints help to reduce the space of possible responses

BASIC PROMPTING TECHNIQUES

- Zero-shot prompting
- One-shot prompting
- Few-shot prompting
- Instruction prompting
- Role-based prompting
- Context-based prompting

ZERO-SHOT PROMPTING

- **Zero-shot prompting** means asking a model to perform a task without providing task-specific examples.
- It asks the model to perform a task with no examples.
- The model relies entirely on its training.
- Example:
 1. **"Classify this review as Positive, Negative, or Neutral: 'The laptop is powerful, but its battery lasts only two hours.'"**
 2. **"Translate this sentence to French: The sky is blue."**

ONE-SHOT PROMPTING

Here, we provide **one example**.

Example

"I loved this movie." → Positive

Classify:

"The movie was fantastic."

The first sentence teaches the AI the expected pattern

FEW-SHOT PROMPTING

- Few-shot prompting provides examples before the actual task.
- Few-shot prompting is particularly useful when the model needs to learn a **specific pattern or output convention from the prompt**.
- Example:
 - Review: "The phone is excellent."
Category: Positive
 - Review: "The application crashes constantly."
Category: Negative
 - Review: "The product arrived yesterday."
Category: Neutral
 - Review: "The battery is fantastic."
Category: ?

ZERO-SHOT VS FEW-SHOT

Feature	Zero-Shot	Few-Shot
Examples	None	Provided
Prompt length	Short	Longer
Setup	Easy	Requires examples
Token usage	Lower	Higher
Complex tasks	Sometimes weaker	Often useful
Format control	Moderate	Often stronger

INSTRUCTION-BASED PROMPTING

Instead of simply giving a topic, we explicitly tell the model **what task to perform.**

Basic prompt:

Artificial Intelligence

Instruction prompt:

Explain Artificial Intelligence in simple language in 5 sentences.

The second prompt gives a clear task.

ROLE-BASED PROMPTING

- Role-based prompting establishes a perspective, responsibility, or expertise level for the model.
- Role prompting is useful because it establishes **context and expectations**.
- A well-designed role prompt can tell the model **what aspects of the code to inspect**.
- Example:
 1. **"Act as a cybersecurity instructor teaching second-year CSE students. Explain SQL injection using a simple vulnerable query and then demonstrate how parameterized queries prevent the attack."**
 2. **"Act as a senior code reviewer. Review the following Python function for correctness, edge cases, complexity, and readability. Explain each problem before suggesting changes."**

Chain-of-Thought (CoT) Prompting

Chain-of-Thought prompting encourages the model to approach a complex problem through **intermediate reasoning steps**.

For example:

Solve this step by step:

If a student has 5 subjects and scores 80, 70, 90, 60 and 100, calculate the average.

Instead of immediately producing the answer, the model can work through:

$$80 + 70 + 90 + 60 + 100 = 400$$

$$400 / 5 = 80$$

Answer = **80**

STRUCTURED PROMPTING

- A useful classroom framework is: **R-T-C-C-O**
 - **R: Role** → Who should the model act as?
 - **T: Task** → What should it do?
 - **C: Context** → What information does it need?
 - **C: Constraints** → What limitations apply?
 - **O: Output** → What should the result look like?
- **Example**
 - Role:** Database instructor
 - Task:** Explain normalization
 - Context:** Third-year CSE students
 - Constraints:** Use one relational example
 - Output:** Definition → example → summary

REASONING-ORIENTED PROMPTING

- Some tasks require multiple reasoning steps.
- Examples:
 - Mathematical problems
 - Logic
 - Algorithm analysis
 - Planning
 - Multi-step decisions
- A prompt can request a **solution approach and verification**.
- Example:
"Solve the problem carefully. State the relevant formula, substitute the values, calculate the result, and verify the final answer."

CONTEXT-BASED PROMPTING

Context gives the AI information about the situation.

Without context:

Write an email.

With context:

I am a college professor. Write an email informing third-year students that tomorrow's lecture will start at 10:30 AM in the seminar hall.

The second prompt gives the AI enough background to produce a more relevant response.

HALLUCINATIONS

- A **hallucination** is an AI-generated claim that is inaccurate, unsupported, or fabricated while potentially appearing plausible.

- Example:

"Give me five research papers published by Professor XYZ on a fictional topic."

- A model might generate realistic-looking:
 - Paper titles
 - Authors
 - Conferences
 - Years
 - DOI-like identifiers
- **Confidence of presentation is not evidence of correctness.**

OUTPUT FORMAT

We can tell AI exactly how we want the answer.

Example

Compare AI, ML and DL in a table.

Instead of: Explain AI, ML and DL.

Possible formats:

- Table
- Bullet points
- JSON
- Steps
- Email
- Report
- Code
- Quiz
- MCQs

The same information can be requested in different formats.

Prompt Refinement

A very important skill is **improving a prompt step by step.**

Prompt 1

Explain AI.

Prompt 2

Explain AI in simple language.

Prompt 3

Explain AI in simple language with 3 real-life examples.

Prompt 4

Explain AI to third-year engineering students using 3 real-life examples and a comparison with human intelligence.

Prompt 5

Explain AI to third-year engineering students in 150 words. Use simple language, 3 real-life examples, and finish with a one-line definition. Students can see how the quality and usefulness of the response can improve as the prompt becomes more precise.

Best Practices for Effective Prompts

1. Be clear

✗ "Tell me something about AI."

✓ "Explain Generative AI in simple language."

2. Give context

"I am preparing a 5-minute classroom presentation..."

3. Specify audience

"Explain this to a beginner."

4. Specify output format

"Give the answer in a table."

5. Give constraints

"Use maximum 100 words."

6. Give examples when necessary

Use few-shot prompting when the desired pattern is difficult to explain.

7. Break complex tasks into steps

Instead of asking:

"Create my entire research report."

Break it into:

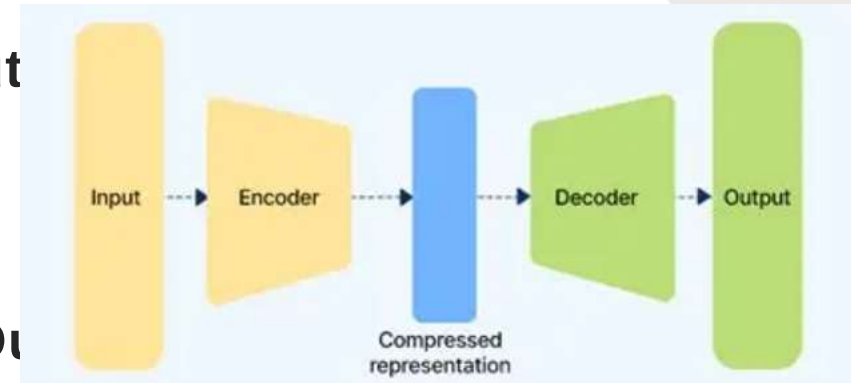
1. Generate title
2. Generate objectives
3. Generate literature review structure
4. Generate methodology
5. Generate conclusion

8. Refine the prompt

First response isn't always the final response.

ENCODER-DECODER ARCHITECTURE

- Now we move from **how we communicate with AI** to **how models process information**.
- An encoder-decoder architecture has two major components:
- **Encoder-----Understands / represents the input**
- **Decoder-----Generates the output.**
- Simple diagram:
- **Input → Encoder → Representation → Decoder → Output**



Encoder

1. Takes the input sequence (text, image features, audio, etc.)
2. Processes it into a **fixed-length or variable-length context representation** (also called a latent vector or hidden state).
3. Captures semantic meaning and dependencies in the input.

Decoder

- Takes the encoder's context representation.
- Generates the output sequence **step-by-step** (autoregressively in many cases).
- Uses previously generated tokens as additional input for the next step

WHY ENCODER-DECODER?

- Because many tasks have:
Input → Output

<i>Input</i>	<i>Output</i>
<i>English sentence</i>	<i>French sentence</i>
<i>Long article</i>	<i>Summary</i>
<i>Question</i>	<i>Answer</i>
<i>Image</i>	<i>Caption</i>
<i>Speech</i>	<i>Text</i>

Encoder: Student listens and understands the English lecture.

Internal representation: Student forms an understanding of the meaning.

Decoder: Student expresses that understanding in Marathi.

Listen → Understand → Express

TRANSFORMER ARCHITECTURE

- The **Transformer** is a neural network architecture that became extremely important in modern NLP and Generative AI.
- The original Transformer architecture was introduced for sequence-to-sequence tasks and uses **attention** as a central mechanism.
- Modern models such as GPT and BERT are based on Transformer architecture.

WHY DO WE NEED TRANSFORMERS?

Consider:

"The cat that was sitting near the window jumped because **it** heard a noise."

What does "**it**" refer to?

Probably the cat.

A model needs to understand relationships between words that may be far apart.

Transformers use **attention mechanisms** to determine which tokens are important to each other.

ATTENTION MECHANISM

Simple definition

Attention allows a model to **focus on relevant parts of the input when processing a particular token.**

Human analogy

Imagine you are reading:

"SARA went to the Classroom because she had to conduct a GenAI lecture."

When understanding "**she**", you naturally look back at "**SARA.**"

This is similar to the idea behind attention.

SELF-ATTENTION

Self-attention means that tokens in the **same sequence look at other tokens in that sequence** to understand their relationships.

Consider:

"The student opened the laptop because it was needed for the lab."

When processing:

"it"

the model may pay attention to:

"laptop"

because the relationship is meaningful.

Simple idea

Self-attention = Every word can look at other words to understand context.

"The Bank was besides the River."

MULTI-HEAD ATTENTION

Instead of using only one attention mechanism, Transformers use **multiple attention heads**.

Each head can learn different relationships.

Consider:

"The student opened the laptop because it was needed for the experiment."

Different attention heads might learn different relationships:

Head 1:

student ↔ opened

Head 2:

laptop ↔ needed

Head 3:

experiment ↔ needed

The outputs of multiple heads are combined.

Imagine **five students** reading the same paragraph.

QUERY, KEY AND VALUE

Self-attention uses three important concepts:

Query (Q)

What am I looking for?

Key (K)

What information do I contain / what can I be matched on?

Value (V)

What information should I provide if selected?

Simple analogy: Library

Imagine searching for a book.

Query:

"I want a book about AI."

Keys:

Book titles/topics used to determine which books match.

Values:

The actual information/content from the selected books.

The model compares the query with keys and uses the values associated with relevant tokens.

WHY SOFTMAX?

- Softmax converts scores into values that behave like weights/probabilities.
- Example:
- Suppose attention scores are:

Word	Score
cat	2.0
laptop	0.5
laboratory	1.0

After Softmax, the model obtains normalized weights indicating how strongly to focus on each word. The largest relevant score gets greater attention.

ATTENTION SCORE

- The model calculates how strongly one token should pay attention to another.
- The standard scaled dot-product attention is:
- $Q + K \rightarrow \text{Attention Scores} \rightarrow \text{Softmax} \rightarrow \text{Weighted } V \rightarrow \text{Output}$

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

POSITIONAL INFORMATION

There is another problem.

Consider:

Dog bites man.

and:

Man bites dog.

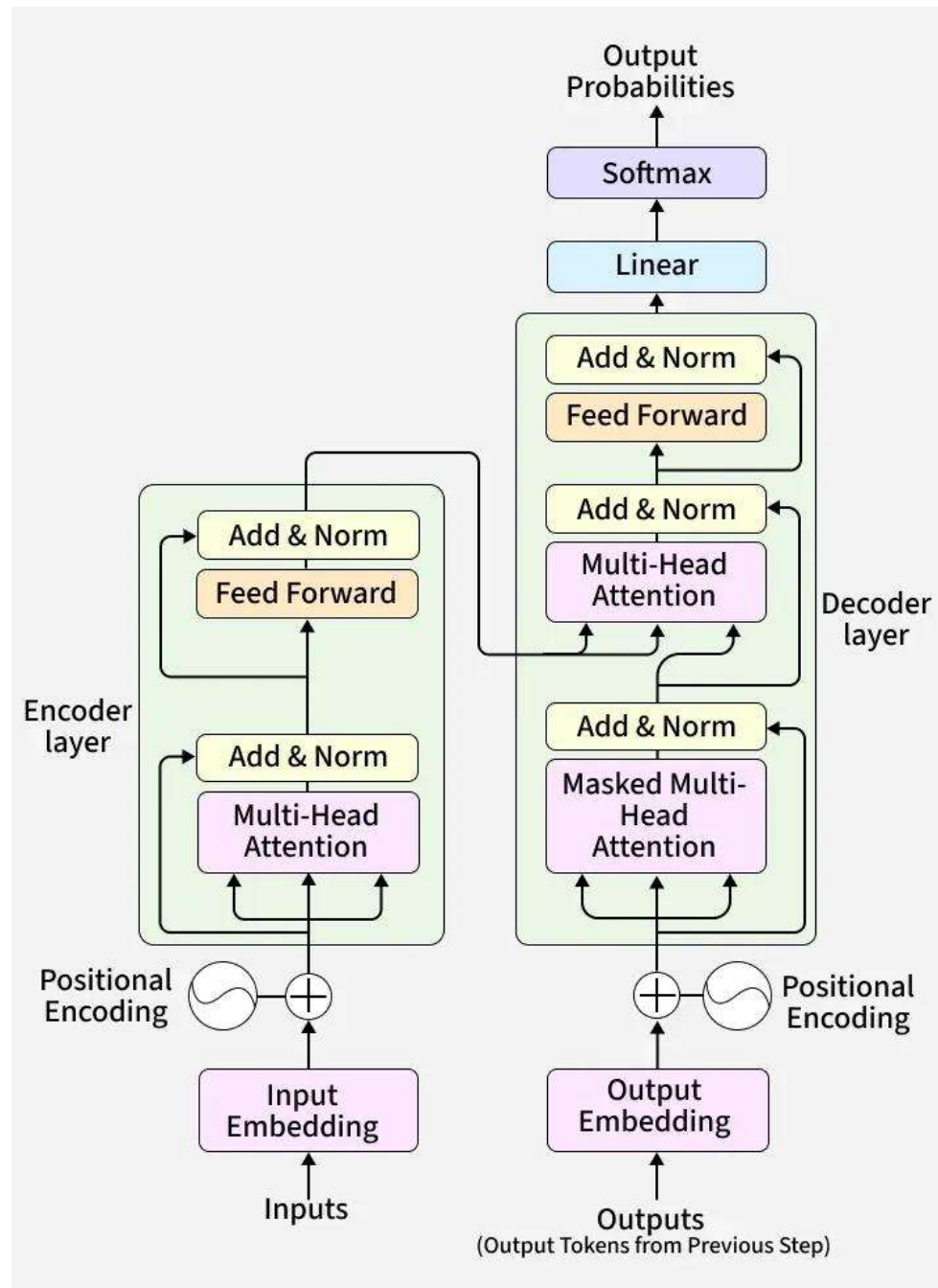
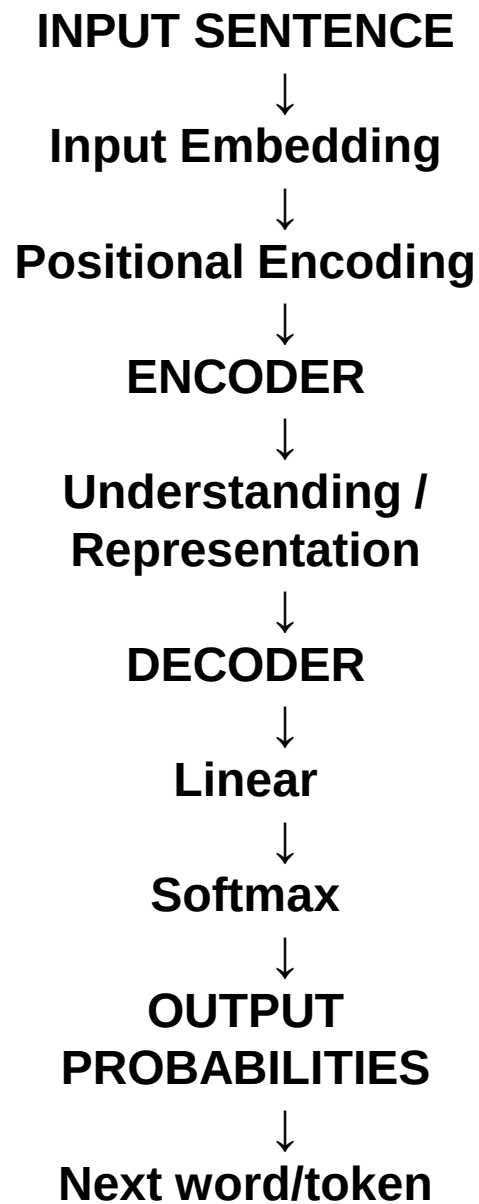
Both sentences contain the same three words.

But the meaning is completely different.

Therefore, the model needs information about:

Where each token occurs in the sequence.

This is called **positional information**.



GPT

- Generative Pre-trained Transformer
- GPT-style models are primarily **decoder-only Transformer models**.
- They are designed for autoregressive generation

How Does GPT Generate Text?

Suppose we give:

"The sky is"

The model predicts a likely next token such as:

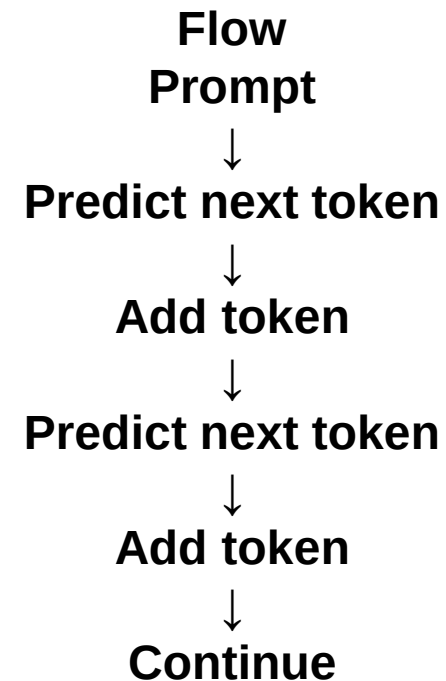
"blue"

Now:

"The sky is blue"

The model predicts the next token.

This process continues.



AUTOREGRESSIVE GENERATION

Autoregressive means:

The model uses previously generated tokens to help generate the next token.

Example:

I



I am



I am learning



I am learning Generative



I am learning Generative AI.

Each new token depends on the previous content

BERT -

BERT stands for:

Bidirectional Encoder Representations from Transformers

BERT is primarily an **encoder-based Transformer model**.

It is mainly designed for **understanding language**.

Why is BERT Called Bidirectional?

Applications of BERT

BERT can be used for:

- Sentiment classification
- Text classification
- Question answering
- Named entity recognition
- Sentence understanding
- Information extraction

GPT VS BERT

Feature	GPT	BERT
Full form	Generative Pre-trained Transformer	Bidirectional Encoder Representations from Transformers
Architecture	Decoder-only	Encoder-only
Main purpose	Generation	Understanding
Processing	Autoregressive	Bidirectional contextual representation
Example	Generate a story	Sentiment classification
Main strength	Producing text	Understanding text

BERT

Input → Encoder → Understanding

Encoder-Decoder

Input → Encoder → Decoder → Output

GPT

Input → Decoder → Generation

TRADE-OFFS IN MODEL DESIGN

A **trade-off** means:

Improving one property may result in a disadvantage somewhere else.

There is no single perfect model.

Model Size vs Computational Cost

Larger models generally have more parameters.

Advantages:

- Greater capacity
- Can model complex patterns
- Often stronger performance

Disadvantages:

- More memory
- More computation
- More expensive training/inference

Example

A small model may run comfortably on a laptop.

A very large model may require specialized hardware.

Quality vs Speed

A more sophisticated model may provide better output but can require more computation.

So we often have:

Higher quality ↔ Higher computational cost

For a mobile application, a slightly smaller model may be preferred if fast response is more important.

Accuracy vs Efficiency

Suppose two models perform:

Model A: 95% accuracy

Model B: 92% accuracy

But:

- Model A requires a powerful GPU.
- Model B runs efficiently on a mobile device.

For a smartphone application, Model B may be the better choice.

Therefore:

Best model $\neq$ always the most accurate model

Context Length vs Resources

A model capable of processing very long input can consider more information.

But longer context can increase:

- Memory requirements
- Computational cost
- Processing time

Example

Compare:

Summarize a 2-page article.

with:

Analyze a 500-page document.

The second task requires substantially more processing

Generalization vs Specialization

General-purpose model

Designed to work across many domains.

Examples:

- Education
- Coding
- Writing
- General Q&A

Specialized model

Designed for a particular domain.

Examples:

- Medical AI
- Legal AI
- Financial AI

A specialized model may perform particularly well on its target domain but may not be as broadly useful.

Interpretability vs Complexity

Simple models can often be easier to understand.

Complex deep learning models can achieve strong performance but may be difficult to interpret.

Therefore:

Performance ↔ Interpretability

can sometimes be an important design consideration.

MINI ACTIVITY

1. Improve these 3 prompts:
 1. "Tell me about AI."
 2. "Write something about climate."
 3. "Explain coding to me."

Discuss: What is missing? Add role, context, task, constraints, and format to each!